AGV 算法可复用母模块

agv_algorithm_mother_modules_v2.py

源码行数：1051 行

排版方式：A4 竖向紧凑版、带行号、自动软换行、保留中文注释与缩进

颜色提示：绿色=注释，蓝色=class/def，紫色=import/装饰器

说明：软换行只影响纸面显示，不修改原始 .py 文件。

生成日期：2026-07-19

打印建议

A4、竖向、双面、实际大小/100%，不要选择“每张多页”。

需要修改或运行时仍使用原始 .py 文件；PDF只用于阅读、批注和课堂打印。

```
"""
AGV 智能调度算法可复用母模块 V2（单文件教学版）
================================================

目标：
1. 用一套固定接口容纳规则、精确优化、元启发式、强化学习、多智能体和混合算法。
2. 提供一个不依赖第三方包、可以直接运行的基础组合：
   规则派工 + Dijkstra 最短路 + 约束过滤 + 时空预约 + 安全校验 + 评估。
3. 把“环境、决策、经验、学习、路径、冲突、安全、评估”拆成可替换模块。

运行：python agv_algorithm_mother_modules_v2.py
测试：python -m unittest -v test_agv_algorithm_mother_modules_v2.py

说明：这是一份母框架，不是某一家工厂可以直接上线的 WCS/WES。
真实项目还要接入地图、PLC/调度接口、订单流、充电规则、故障处理和数据库。
"""

from __future__ import annotations

from abc import ABC, abstractmethod
from collections import defaultdict, deque
from dataclasses import dataclass, field, replace
from heapq import heappop, heappush
import math
import random
from typing import Any, Callable, Hashable, Iterable, Sequence


# ============================================================================
# 第 1 层：问题定义与状态 —— “要解决什么”
# ============================================================================


@dataclass(frozen=True)
class AGV:
    """一台 AGV 在某个决策时刻的状态。"""

    agv_id: str
    node: str
    battery: float
    capacity: float
    busy: bool = False
    faulted: bool = False


@dataclass(frozen=True)
class Task:
    """一个运输任务。priority 越大，越希望优先处理。"""

    task_id: str
    pickup: str
    dropoff: str
    load: float
    priority: float = 1.0
    assigned: bool = False
    due_time: int | None = None


@dataclass(frozen=True)
class SchedulingSnapshot:
    """一次决策看到的完整快照；对应强化学习中的 observation/state。"""

    time: int
    agvs: tuple[AGV, ...]
    tasks: tuple[Task, ...]
    metadata: dict[str, Any] = field(default_factory=dict)

    def get_agv(self, agv_id: str) -> AGV:
        return next(x for x in self.agvs if x.agv_id == agv_id)

    def get_task(self, task_id: str) -> Task:
        return next(x for x in self.tasks if x.task_id == task_id)


class WeightedGraph:
    """轻量有权图；可替换为真实路网、NetworkX 或数字孪生地图服务。"""

    def __init__(self) -> None:
        self._adj: dict[str, list[tuple[str, float]]] = defaultdict(list)

    def add_edge(self, a: str, b: str, cost: float = 1.0, bidirectional: bool = True) -> None:
        if cost <= 0:
            raise ValueError("边权必须大于 0")
        self._adj[a].append((b, float(cost)))
        self._adj.setdefault(b, [])
        if bidirectional:
            self._adj[b].append((a, float(cost)))

    def neighbors(self, node: str) -> tuple[tuple[str, float], ...]:
        return tuple(self._adj.get(node, ()))

    def shortest_path(
        self,
        start: str,
```

```python
        goal: str,
        heuristic: Callable[[str, str], float] | None = None,
    ) -> tuple[list[str], float]:
        """Dijkstra; 传入可采纳 heuristic 时就是 A* 的母模板。"""
        if start == goal:
            return [start], 0.0
        h = heuristic or (lambda _a, _b: 0.0)
        frontier: list[tuple[float, float, str]] = [(h(start, goal), 0.0, start)]
        previous: dict[str, str | None] = {start: None}
        best_g: dict[str, float] = {start: 0.0}

        while frontier:
            _f, g, node = heappop(frontier)
            if g != best_g.get(node):
                continue
            if node == goal:
                path: list[str] = []
                cursor: str | None = goal
                while cursor is not None:
                    path.append(cursor)
                    cursor = previous[cursor]
                return list(reversed(path)), g
            for nxt, edge_cost in self.neighbors(node):
                new_g = g + edge_cost
                if new_g < best_g.get(nxt, math.inf):
                    best_g[nxt] = new_g
                    previous[nxt] = node
                    heappush(frontier, (new_g + h(nxt, goal), new_g, nxt))
        raise ValueError(f"节点 {start!r} 与 {goal!r} 之间不存在可行路径")


@dataclass(frozen=True)
class ProblemDefinition:
    """项目级固定配置。新的 AGV 项目主要替换这里和状态构造器。"""

    graph: WeightedGraph
    min_battery: float = 15.0
    energy_per_distance: float = 1.0
    max_wait_steps: int = 50
    weights: dict[str, float] = field(
        default_factory=lambda: {
            "travel": 1.0,
            "tardiness": 2.0,
            "energy": 0.2,
            "priority": 1.0,
        }
    )


class StateBuilder:
    """把数据库/数字孪生/仿真器原始数据整理为统一快照。"""

    def build(
        self,
        time: int,
        agvs: Iterable[AGV],
        tasks: Iterable[Task],
        metadata: dict[str, Any] | None = None,
    ) -> SchedulingSnapshot:
        return SchedulingSnapshot(time, tuple(agvs), tuple(tasks), metadata or {})


class FeaturePipeline:
    """
    感知/预测/表征母模块。

    YOLO、LSTM、Transformer、GNN 等通常只是流水线中的一个变换，
    最终输出仍要交给 DecisionEngine，而不是直接绕过约束和安全层。
    """

    def __init__(self, transforms: Sequence[Callable[[Any], Any]]) -> None:
        self.transforms = tuple(transforms)

    def transform(self, raw_input: Any) -> Any:
        value = raw_input
        for transform in self.transforms:
            value = transform(value)
        return value


# ============================================================================
# 第 2 层：候选、约束与目标 —— “哪些能做、哪个好”
# ============================================================================


@dataclass(frozen=True)
class CandidateDecision:
    agv_id: str
    task_id: str
    score: float = math.inf
    explanation: str = ""


class CandidateGenerator:
```

```python
    """生成 AGV × 任务候选对；复杂项目可加入候选任务窗口或邻域剪枝。"""

    def generate(self, snapshot: SchedulingSnapshot) -> list[CandidateDecision]:
        agvs = [a for a in snapshot.agvs if not a.busy and not a.faulted]
        tasks = [t for t in snapshot.tasks if not t.assigned]
        return [CandidateDecision(a.agv_id, t.task_id) for a in agvs for t in tasks]


class ConstraintChecker:
    """硬约束过滤器：违反即不能执行，不应只靠负奖励碰运气。"""

    def is_feasible(
        self,
        problem: ProblemDefinition,
        snapshot: SchedulingSnapshot,
        candidate: CandidateDecision,
    ) -> tuple[bool, str]:
        agv = snapshot.get_agv(candidate.agv_id)
        task = snapshot.get_task(candidate.task_id)
        if agv.busy or agv.faulted:
            return False, "AGV 忙碌或故障"
        if task.assigned:
            return False, "任务已分配"
        if task.load > agv.capacity:
            return False, "载荷超过容量"
        try:
            _, to_pickup = problem.graph.shortest_path(agv.node, task.pickup)
            _, to_dropoff = problem.graph.shortest_path(task.pickup, task.dropoff)
        except ValueError:
            return False, "路网不连通"
        predicted_battery = agv.battery - (to_pickup + to_dropoff) * problem.energy_per_distance
        if predicted_battery < problem.min_battery:
            return False, "预计执行后电量低于安全阈值"
        return True, "可行"

    def filter(
        self,
        problem: ProblemDefinition,
        snapshot: SchedulingSnapshot,
        candidates: Iterable[CandidateDecision],
    ) -> list[CandidateDecision]:
        return [c for c in candidates if self.is_feasible(problem, snapshot, c)[0]]


@dataclass(frozen=True)
class ActionMask:
    """把硬约束转换成策略能使用的合法动作掩码。"""

    allowed: tuple[bool, ...]

    @classmethod
    def from_keys(
        cls,
        all_action_keys: Sequence[Hashable],
        feasible_action_keys: set[Hashable],
    ) -> "ActionMask":
        return cls(tuple(key in feasible_action_keys for key in all_action_keys))

    @classmethod
    def from_candidates(
        cls,
        all_candidates: Sequence[CandidateDecision],
        feasible_candidates: Sequence[CandidateDecision],
    ) -> "ActionMask":
        feasible = {(x.agv_id, x.task_id) for x in feasible_candidates}
        keys = [(x.agv_id, x.task_id) for x in all_candidates]
        return cls.from_keys(keys, feasible)

    def legal_indices(self) -> list[int]:
        return [i for i, allowed in enumerate(self.allowed) if allowed]

    def validate(self, action_index: int) -> None:
        if action_index < 0 or action_index >= len(self.allowed):
            raise ValueError(f"动作索引 {action_index} 越界")
        if not self.allowed[action_index]:
            raise ValueError(f"动作索引 {action_index} 被硬约束屏蔽")


class ObjectiveEvaluator:
    """统一目标函数：规则、优化和学习算法都用同一评价口径。"""

    def score(
        self,
        problem: ProblemDefinition,
        snapshot: SchedulingSnapshot,
        candidate: CandidateDecision,
    ) -> CandidateDecision:
        agv = snapshot.get_agv(candidate.agv_id)
        task = snapshot.get_task(candidate.task_id)
        _, empty_distance = problem.graph.shortest_path(agv.node, task.pickup)
        _, loaded_distance = problem.graph.shortest_path(task.pickup, task.dropoff)
        travel = empty_distance + loaded_distance
        energy = travel * problem.energy_per_distance
        tardiness = 0.0
```

```python
        if task.due_time is not None:
            tardiness = max(0.0, snapshot.time + travel - task.due_time)
        w = problem.weights
        total = (
            w.get("travel", 1.0) * travel
            + w.get("energy", 0.0) * energy
            + w.get("tardiness", 0.0) * tardiness
            - w.get("priority", 0.0) * task.priority
        )
        reason = (
            f"空驶={empty_distance:.1f}, 载货={loaded_distance:.1f}, "
            f"能耗={energy:.1f}, 延误={tardiness:.1f}, 优先级={task.priority:.1f}"
        )
        return replace(candidate, score=total, explanation=reason)


class RiskEvaluator:
    """随机/鲁棒/风险敏感优化共用的场景代价聚合器。"""

    def aggregate(
        self,
        scenario_costs: Sequence[float],
        mode: str = "mean",
        alpha: float = 0.9,
    ) -> float:
        if not scenario_costs:
            raise ValueError("scenario_costs 不能为空")
        costs = sorted(float(x) for x in scenario_costs)
        if mode == "mean":
            return sum(costs) / len(costs)
        if mode == "worst":
            return costs[-1]
        if mode == "cvar":
            if not 0 <= alpha < 1:
                raise ValueError("CVaR alpha 必须位于 [0, 1)")
            tail_size = max(1, math.ceil((1.0 - alpha) * len(costs)))
            tail = costs[-tail_size:]
            return sum(tail) / len(tail)
        raise ValueError("mode 只支持 mean、worst 或 cvar")


class MultiObjectiveEvaluator:
    """多目标优化的Pareto前沿母模块。"""

    def __init__(self, minimize: Sequence[str] = (), maximize: Sequence[str] = ()) -> None:
        if not minimize and not maximize:
            raise ValueError("至少指定一个最小化或最大化目标")
        self.minimize = tuple(minimize)
        self.maximize = tuple(maximize)

    def dominates(self, left: dict[str, Any], right: dict[str, Any]) -> bool:
        no_worse = True
        strictly_better = False
        for name in self.minimize:
            no_worse &= left[name] <= right[name]
            strictly_better |= left[name] < right[name]
        for name in self.maximize:
            no_worse &= left[name] >= right[name]
            strictly_better |= left[name] > right[name]
        return bool(no_worse and strictly_better)

    def pareto_front(self, records: Sequence[dict[str, Any]]) -> list[dict[str, Any]]:
        return [
            record
            for i, record in enumerate(records)
            if not any(
                i != j and self.dominates(other, record)
                for j, other in enumerate(records)
            )
        ]


# ============================================================================
# 第 3 层：决策引擎 —— “由哪类算法做选择”
# ============================================================================


class DecisionEngine(ABC):
    """所有算法共同遵守的母接口。"""

    name = "decision-engine"

    def reset(self, problem: ProblemDefinition) -> None:
        """新实验开始时清理内部状态；无状态算法可不做任何事。"""

    @abstractmethod
    def decide(
        self,
        problem: ProblemDefinition,
        snapshot: SchedulingSnapshot,
        candidates: Sequence[CandidateDecision],
    ) -> CandidateDecision:
        raise NotImplementedError
```

```python
    def update(self, feedback: Any) -> None:
        """在线学习或滚动优化可接收反馈；静态规则可忽略。"""


class RuleDecisionEngine(DecisionEngine):
    """可解释基线：计算所有候选分数并取最小值。"""

    name = "weighted-nearest-rule"

    def __init__(self, evaluator: ObjectiveEvaluator | None = None) -> None:
        self.evaluator = evaluator or ObjectiveEvaluator()

    def decide(self, problem, snapshot, candidates):
        if not candidates:
            raise RuntimeError("当前没有可行的 AGV—任务候选")
        scored = [self.evaluator.score(problem, snapshot, c) for c in candidates]
        return min(scored, key=lambda x: (x.score, x.agv_id, x.task_id))


class ExactSolverAdapter(DecisionEngine):
    """
    MILP/CP-SAT/网络流等精确优化的适配母类。

    子类只需要把统一问题翻译给求解器，并将解翻译回 CandidateDecision。
    真实项目可分别实现 OrToolsCPSATEngine、GurobiMILPEngine 等。
    """

    name = "exact-solver-adapter"

    @abstractmethod
    def build_model(self, problem, snapshot, candidates) -> Any:
        raise NotImplementedError

    @abstractmethod
    def solve_model(self, model: Any) -> Any:
        raise NotImplementedError

    @abstractmethod
    def decode_solution(self, raw_solution: Any) -> CandidateDecision:
        raise NotImplementedError

    def decide(self, problem, snapshot, candidates):
        model = self.build_model(problem, snapshot, candidates)
        return self.decode_solution(self.solve_model(model))


class MetaheuristicEngine:
    """GA/PSO/ACO/VNS/ALNS 等可共用的“解—评价—扰动—接受”母循环。"""

    def __init__(self, iterations: int = 100, seed: int = 0) -> None:
        self.iterations = iterations
        self.rng = random.Random(seed)

    def optimize(
        self,
        initial_solution: Any,
        objective: Callable[[Any], float],
        neighbor: Callable[[Any, random.Random], Any],
        repair: Callable[[Any], Any] | None = None,
        accept: Callable[[float, float, int], bool] | None = None,
    ) -> tuple[Any, float]:
        repair_fn = repair or (lambda x: x)
        accept_fn = accept or (lambda new, old, _i: new <= old)
        current = repair_fn(initial_solution)
        current_score = objective(current)
        best, best_score = current, current_score
        for i in range(self.iterations):
            trial = repair_fn(neighbor(current, self.rng))
            trial_score = objective(trial)
            if accept_fn(trial_score, current_score, i):
                current, current_score = trial, trial_score
            if trial_score < best_score:
                best, best_score = trial, trial_score
        return best, best_score


class RLDecisionAdapter(DecisionEngine):
    """
    PPO/DQN/SAC 等模型无关强化学习的统一适配器。

    encoder: SchedulingSnapshot -> 算法需要的 observation
    predictor: observation, valid_action_ids -> action_id
    decoder: action_id, candidates -> CandidateDecision
    """

    name = "rl-policy-adapter"

    def __init__(
        self,
        encoder: Callable[[SchedulingSnapshot], Any],
        predictor: Callable[[Any, list[int]], int],
    ) -> None:
        self.encoder = encoder
        self.predictor = predictor
```

```python

    def decide(self, problem, snapshot, candidates):
        if not candidates:
            raise RuntimeError("没有可行动作")
        observation = self.encoder(snapshot)
        valid_action_ids = list(range(len(candidates)))  # action mask 的最小形式
        action_id = self.predictor(observation, valid_action_ids)
        if action_id not in valid_action_ids:
            raise RuntimeError(f"策略返回了非法动作 {action_id}")
        return candidates[action_id]


class MARLDecisionAdapter(DecisionEngine):
    """IQL/QMIX/MADDPG/MAPPO 等多智能体算法的联合动作适配母类。"""

    name = "marl-adapter"

    @abstractmethod
    def decide_jointly(
        self, problem: ProblemDefinition, snapshot: SchedulingSnapshot
    ) -> dict[str, Any]:
        raise NotImplementedError

    def decide(self, problem, snapshot, candidates):
        joint = self.decide_jointly(problem, snapshot)
        agv_id = str(joint["agv_id"])
        task_id = str(joint["task_id"])
        match = next((c for c in candidates if c.agv_id == agv_id and c.task_id == task_id), None)
        if match is None:
            raise RuntimeError("多智能体联合动作不满足当前约束")
        return match


class ModelBasedPlanner:
    """
    模型式强化学习/学习模型+MPC的通用短视域规划器。

    transition_model 预测下一状态，reward_model 评价一步结果；
    planner 在模型中向前滚动，而不是只凭当前策略直接出动作。
    """

    def __init__(
        self,
        action_provider: Callable[[Any], Sequence[Any]],
        transition_model: Callable[[Any, Any], Any],
        reward_model: Callable[[Any, Any, Any], float],
        horizon: int = 3,
        discount: float = 0.99,
        beam_width: int = 16,
    ) -> None:
        if horizon <= 0 or beam_width <= 0:
            raise ValueError("horizon 和 beam_width 必须大于 0")
        self.action_provider = action_provider
        self.transition_model = transition_model
        self.reward_model = reward_model
        self.horizon = horizon
        self.discount = discount
        self.beam_width = beam_width

    def plan(self, initial_state: Any) -> tuple[Any, float, list[Any]]:
        beams: list[tuple[float, Any, list[Any]]] = [(0.0, initial_state, [])]
        for depth in range(self.horizon):
            expanded: list[tuple[float, Any, list[Any]]] = []
            for score, state, sequence in beams:
                for action in self.action_provider(state):
                    next_state = self.transition_model(state, action)
                    reward = self.reward_model(state, action, next_state)
                    total = score + (self.discount**depth) * reward
                    expanded.append((total, next_state, sequence + [action]))
            if not expanded:
                break
            expanded.sort(key=lambda item: item[0], reverse=True)
            beams = expanded[: self.beam_width]
        if not beams or not beams[0][2]:
            raise RuntimeError("模型规划器没有找到可行动作")
        best_score, _state, sequence = beams[0]
        return sequence[0], best_score, sequence


class HierarchicalController:
    """分层强化学习/两阶段调度的最小母接口。"""

    def __init__(
        self,
        high_level_policy: Callable[[Any], Any],
        low_level_policy: Callable[[Any, Any], Any],
    ) -> None:
        self.high_level_policy = high_level_policy
        self.low_level_policy = low_level_policy

    def decide(self, state: Any) -> tuple[Any, Any]:
        subgoal = self.high_level_policy(state)
        primitive_action = self.low_level_policy(state, subgoal)
        return subgoal, primitive_action
```

```python


class GameCoordinator:
    """势博弈/非合作博弈/拍卖前的通用最优响应母循环。"""

    def __init__(
        self,
        players: Sequence[Hashable],
        action_provider: Callable[[Hashable, Any], Sequence[Any]],
        utility: Callable[[Hashable, Any, dict[Hashable, Any], Any], float],
    ) -> None:
        self.players = tuple(players)
        self.action_provider = action_provider
        self.utility = utility

    def best_response(
        self,
        state: Any,
        initial: dict[Hashable, Any],
        rounds: int = 20,
    ) -> dict[Hashable, Any]:
        joint = dict(initial)
        for _ in range(rounds):
            changed = False
            for player in self.players:
                actions = list(self.action_provider(player, state))
                if not actions:
                    continue
                best = max(
                    actions,
                    key=lambda action: (
                        self.utility(player, action, joint, state),
                        str(action),
                    ),
                )
                changed |= joint.get(player) != best
                joint[player] = best
            if not changed:
                break
        return joint


# ============================================================================
# 第 4 层：路径、冲突与安全 —— “怎么安全执行”
# ============================================================================


class PathPlanner:
    """路径规划统一入口；可替换为 A*、SIPP、CBS、D* Lite 或外部服务。"""

    def plan(self, graph: WeightedGraph, start: str, goal: str) -> tuple[list[str], float]:
        return graph.shortest_path(start, goal)


class ReservationTable:
    """简化时空预约表：检查节点冲突和对向边冲突。"""

    def __init__(self) -> None:
        self.nodes: dict[tuple[int, str], str] = {}
        self.edges: dict[tuple[int, str, str], str] = {}

    def is_free(self, path: Sequence[str], start_time: int, owner: str = "candidate") -> bool:
        for i, node in enumerate(path):
            t = start_time + i
            occupant = self.nodes.get((t, node))
            if occupant is not None and occupant != owner:
                return False
            if i > 0:
                prev = path[i - 1]
                opposite = self.edges.get((t, node, prev))
                if opposite is not None and opposite != owner:
                    return False
        return True

    def reserve(self, path: Sequence[str], start_time: int, owner: str) -> None:
        for i, node in enumerate(path):
            t = start_time + i
            self.nodes[(t, node)] = owner
            if i > 0:
                self.edges[(t, path[i - 1], node)] = owner

    def resolve(
        self,
        path: Sequence[str],
        start_time: int,
        max_wait_steps: int = 50,
        owner: str = "candidate",
    ) -> tuple[list[str], int]:
        if not path:
            raise ValueError("路径不能为空")
        for waits in range(max_wait_steps + 1):
            delayed = [path[0]] * waits + list(path)
            if self.is_free(delayed, start_time, owner):
                return delayed, waits
```

```python
        raise RuntimeError("在最大等待步数内无法消解路径冲突")


class ConflictResolver(ReservationTable):
    """预约表的工程封装；以后可由CBS、ECBS或PIBT实现同一职责。"""

    def resolve_and_reserve(
        self,
        path: Sequence[str],
        start_time: int,
        owner: str,
        max_wait_steps: int = 50,
    ) -> tuple[list[str], int]:
        repaired, waits = self.resolve(path, start_time, max_wait_steps, owner)
        self.reserve(repaired, start_time, owner)
        return repaired, waits


@dataclass(frozen=True)
class ExecutionPlan:
    agv_id: str
    task_id: str
    to_pickup: list[str]
    to_dropoff: list[str]
    distance: float
    wait_steps: int = 0
    safe: bool = False
    explanation: str = ""

    @property
    def full_path(self) -> list[str]:
        if not self.to_pickup:
            return list(self.to_dropoff)
        return list(self.to_pickup) + list(self.to_dropoff[1:])


class SafetyShield:
    """安全盾：最终动作执行前再做确定性校验。"""

    def validate(
        self, problem: ProblemDefinition, snapshot: SchedulingSnapshot, plan: ExecutionPlan
    ) -> tuple[bool, str]:
        agv = snapshot.get_agv(plan.agv_id)
        task = snapshot.get_task(plan.task_id)
        if agv.faulted or agv.busy:
            return False, "AGV 状态不可执行"
        if task.load > agv.capacity:
            return False, "载荷超限"
        remaining = agv.battery - plan.distance * problem.energy_per_distance
        if remaining < problem.min_battery:
            return False, "执行后电量越过安全线"
        if plan.full_path[0] != agv.node or plan.full_path[-1] != task.dropoff:
            return False, "路径起终点与任务不一致"
        return True, "安全校验通过"


# ============================================================================
# 第 5 层：经验与学习 —— “算法从什么数据中更新”
# ============================================================================


@dataclass(frozen=True)
class Transition:
    state: Any
    action: Any
    reward: float
    next_state: Any
    done: bool
    info: dict[str, Any] = field(default_factory=dict)


class ExperienceModule(ABC):
    @abstractmethod
    def add(self, transition: Transition) -> None:
        raise NotImplementedError

    @abstractmethod
    def __len__(self) -> int:
        raise NotImplementedError


class ReplayBuffer(ExperienceModule):
    """DQN/DDPG/TD3/SAC 常用的离策略经验回放。"""

    def __init__(self, capacity: int, seed: int = 0) -> None:
        if capacity <= 0:
            raise ValueError("capacity 必须大于 0")
        self._items: deque[Transition] = deque(maxlen=capacity)
        self._rng = random.Random(seed)

    def add(self, transition: Transition) -> None:
        self._items.append(transition)

    def sample(self, batch_size: int) -> list[Transition]:
```

```python
        if batch_size > len(self._items):
            raise ValueError("样本数不足")
        return self._rng.sample(list(self._items), batch_size)

    def __len__(self) -> int:
        return len(self._items)


class RolloutBuffer(ExperienceModule):
    """PPO/A2C 常用的同策略轨迹缓存；训练一次后通常清空。"""

    def __init__(self) -> None:
        self._items: list[Transition] = []

    def add(self, transition: Transition) -> None:
        self._items.append(transition)

    def consume(self) -> list[Transition]:
        batch, self._items = self._items, []
        return batch

    def __len__(self) -> int:
        return len(self._items)


class LearningModule(ABC):
    @abstractmethod
    def learn(self, experience: ExperienceModule) -> dict[str, float]:
        raise NotImplementedError


class TabularQLearner:
    """Q-learning 的最小可运行内核；用于理解，不是复杂 AGV 的最终算法。"""

    def __init__(self, alpha: float = 0.1, gamma: float = 0.95, seed: int = 0) -> None:
        self.alpha = alpha
        self.gamma = gamma
        self.q: dict[tuple[Hashable, Hashable], float] = defaultdict(float)
        self.rng = random.Random(seed)

    def get_q(self, state: Hashable, action: Hashable) -> float:
        return self.q[(state, action)]

    def set_q(self, state: Hashable, action: Hashable, value: float) -> None:
        self.q[(state, action)] = float(value)

    def choose_action(
        self, state: Hashable, actions: Sequence[Hashable], epsilon: float
    ) -> Hashable:
        if not actions:
            raise ValueError("actions 不能为空")
        if self.rng.random() < epsilon:  # 探索
            return self.rng.choice(list(actions))
        return max(actions, key=lambda a: (self.get_q(state, a), str(a)))  # 利用

    def update(
        self,
        state: Hashable,
        action: Hashable,
        reward: float,
        next_state: Hashable,
        next_actions: Sequence[Hashable],
        done: bool = False,
    ) -> float:
        old_q = self.get_q(state, action)
        future = 0.0 if done or not next_actions else max(self.get_q(next_state, a) for a in next_actions)
        target = reward + self.gamma * future
        td_error = target - old_q
        self.set_q(state, action, old_q + self.alpha * td_error)
        return td_error


# ============================================================================
# 第 6 层：混合协调、执行与评估 —— “把模块串成工程闭环”
# ============================================================================


class HybridCoordinator:
    """上层派工 + 下层路径 + 冲突处理 + 安全盾的可运行组合。"""

    def __init__(
        self,
        decision_engine: DecisionEngine,
        candidate_generator: CandidateGenerator | None = None,
        constraint_checker: ConstraintChecker | None = None,
        path_planner: PathPlanner | None = None,
        conflict_resolver: ReservationTable | None = None,
        safety_shield: SafetyShield | None = None,
    ) -> None:
        self.decision_engine = decision_engine
        self.candidate_generator = candidate_generator or CandidateGenerator()
        self.constraint_checker = constraint_checker or ConstraintChecker()
        self.path_planner = path_planner or PathPlanner()
        self.conflict_resolver = conflict_resolver or ConflictResolver()
```

```python
        self.safety_shield = safety_shield or SafetyShield()

    def plan_one(self, problem: ProblemDefinition, snapshot: SchedulingSnapshot) -> ExecutionPlan:
        raw = self.candidate_generator.generate(snapshot)
        feasible = self.constraint_checker.filter(problem, snapshot, raw)
        decision = self.decision_engine.decide(problem, snapshot, feasible)
        agv = snapshot.get_agv(decision.agv_id)
        task = snapshot.get_task(decision.task_id)

        pickup_path, pickup_cost = self.path_planner.plan(problem.graph, agv.node, task.pickup)
        dropoff_path, dropoff_cost = self.path_planner.plan(problem.graph, task.pickup, task.dropoff)
        original_full = pickup_path + dropoff_path[1:]
        resolved_full, waits = self.conflict_resolver.resolve(
            original_full,
            start_time=snapshot.time,
            max_wait_steps=problem.max_wait_steps,
            owner=agv.agv_id,
        )
        # 等待都加在起点，因此可按取货节点位置重新切分。
        pickup_end = max(i for i, node in enumerate(resolved_full) if node == task.pickup)
        to_pickup = resolved_full[: pickup_end + 1]
        to_dropoff = resolved_full[pickup_end:]
        plan = ExecutionPlan(
            agv_id=agv.agv_id,
            task_id=task.task_id,
            to_pickup=to_pickup,
            to_dropoff=to_dropoff,
            distance=pickup_cost + dropoff_cost,
            wait_steps=waits,
            explanation=decision.explanation,
        )
        safe, safety_reason = self.safety_shield.validate(problem, snapshot, plan)
        if not safe:
            raise RuntimeError(f"安全盾拒绝计划: {safety_reason}")
        plan = replace(plan, safe=True, explanation=f"{plan.explanation}; {safety_reason}")
        self.conflict_resolver.reserve(plan.full_path, snapshot.time, agv.agv_id)
        return plan


class Executor:
    """教学用状态推进器; 真实项目应替换为 WCS/仿真器/数字孪生接口。"""

    def apply(
        self, problem: ProblemDefinition, snapshot: SchedulingSnapshot, plan: ExecutionPlan
    ) -> SchedulingSnapshot:
        agvs = tuple(
            replace(
                a,
                node=snapshot.get_task(plan.task_id).dropoff,
                battery=a.battery - plan.distance * problem.energy_per_distance,
                busy=False,
            )
            if a.agv_id == plan.agv_id
            else a
            for a in snapshot.agvs
        )
        tasks = tuple(t for t in snapshot.tasks if t.task_id != plan.task_id)
        return SchedulingSnapshot(
            time=snapshot.time + len(plan.full_path) - 1,
            agvs=agvs,
            tasks=tasks,
            metadata=dict(snapshot.metadata),
        )


class DigitalTwinAdapter(ABC):
    """数字孪生不是调度算法; 它负责读取实时状态并下发计划。"""

    @abstractmethod
    def read_snapshot(self) -> SchedulingSnapshot:
        raise NotImplementedError

    @abstractmethod
    def dispatch(self, plan: ExecutionPlan) -> None:
        raise NotImplementedError


@dataclass
class EvaluationMetrics:
    completed_tasks: int = 0
    total_distance: float = 0.0
    total_wait_steps: int = 0
    safety_rejections: int = 0
    decision_count: int = 0

    @property
    def average_distance(self) -> float:
        return self.total_distance / self.completed_tasks if self.completed_tasks else 0.0


class Evaluator:
    def __init__(self) -> None:
        self.metrics = EvaluationMetrics()
```

```python
    def record(self, plan: ExecutionPlan) -> None:
        self.metrics.completed_tasks += 1
        self.metrics.decision_count += 1
        self.metrics.total_distance += plan.distance
        self.metrics.total_wait_steps += plan.wait_steps


class ExperimentManager:
    """固定随机种子、算法、场景和评价指标，避免只看单次 reward。"""

    def __init__(self, coordinator: HybridCoordinator, evaluator: Evaluator | None = None) -> None:
        self.coordinator = coordinator
        self.evaluator = evaluator or Evaluator()

    def run_until_empty(
        self,
        problem: ProblemDefinition,
        snapshot: SchedulingSnapshot,
        max_decisions: int = 100,
    ) -> tuple[SchedulingSnapshot, EvaluationMetrics, list[ExecutionPlan]]:
        executor = Executor()
        plans: list[ExecutionPlan] = []
        current = snapshot
        while current.tasks and len(plans) < max_decisions:
            plan = self.coordinator.plan_one(problem, current)
            plans.append(plan)
            self.evaluator.record(plan)
            current = executor.apply(problem, current, plan)
        return current, self.evaluator.metrics, plans


# ============================================================================
# 第 7 层：算法目录 —— “选算法时先看场景，不看热度”
# ============================================================================


@dataclass(frozen=True)
class AlgorithmFamily:
    family: str
    representatives: tuple[str, ...]
    fixed_mother_loop: str
    preferred_scenes: str
    reusable_modules: tuple[str, ...]


def build_algorithm_catalog() -> tuple[AlgorithmFamily, ...]:
    """与打印版 PDF 对应的算法家族索引。"""
    return (
        AlgorithmFamily("派工规则", ("最近车", "最早可用", "EDD", "SPT", "组合优先级"), "生成候选→过滤→打分→取最优", "实时性强、规则明确、基线与兜底", ("CandidateGenerator", "ConstraintChecker", "ObjectiveEvaluator")),
        AlgorithmFamily("精确优化", ("MILP", "MINLP", "CP-SAT", "网络流", "Benders", "列生成"), "变量→目标→约束→求解→解码", "中小规模、要求最优性界或严谨约束", ("ProblemDefinition", "ExactSolverAdapter")),
        AlgorithmFamily("不确定与多目标优化", ("随机规划", "鲁棒优化", "DRO", "CVaR", "NSGA-II", "MOEA/D"), "场景/风险/目标建模→求解→Pareto选择", "到达随机、时长波动、能耗与效率权衡", ("ObjectiveEvaluator", "ExperimentManager")),
        AlgorithmFamily("单车路径规划", ("Dijkstra", "A*", "D* Lite", "LPA*", "JPS", "Theta*", "SIPP"), "开放集→扩展→代价更新→回溯", "静态/动态路网中的单车最短安全路径", ("WeightedGraph", "PathPlanner")),
        AlgorithmFamily("多车冲突与 MAPF", ("预约表", "Cooperative A*", "WHCA*", "CBS", "ECBS", "PIBT", "M*"), "路径→检测冲突→加约束/优先级→重规划", "多 AGV 节点、边和死锁冲突", ("ReservationTable", "SafetyShield")),
        AlgorithmFamily("元启发式", ("GA", "PSO", "ACO", "SA", "TS", "VNS", "ALNS", "FOA", "FPA"), "编码→评价→搜索算子→接受→终止", "组合爆炸、混合约束、需要较好可行解", ("MetaheuristicEngine", "ConstraintChecker")),
        AlgorithmFamily("表格强化学习", ("Q-learning", "SARSA", "Expected SARSA"), "交互→TD目标→更新Q值→探索衰减", "小离散状态；教学、小基线", ("TabularQLearner", "ExperienceModule")),
        AlgorithmFamily("深度价值学习", ("DQN", "Double DQN", "Dueling DQN", "PER", "Rainbow"), "回放采样→TD目标→网络更新→目标网络", "离散动作、状态较大", ("ReplayBuffer", "RLDecisionAdapter")),
        AlgorithmFamily("策略梯度/Actor-Critic", ("REINFORCE", "A2C", "PPO", "DDPG", "TD3", "SAC"), "采样→优势/回报→策略与价值更新", "PPO离散/连续均可；SAC/TD3偏连续控制", ("RolloutBuffer", "RLDecisionAdapter")),
        AlgorithmFamily("模型式强化学习", ("Dyna-Q", "PETS", "MBPO", "Dreamer", "MuZero", "学习模型+MPC"), "学习转移模型→虚拟滚动→规划/策略更新", "真实试错昂贵、可利用仿真或数字孪生", ("DigitalTwinAdapter", "HybridCoordinator")),
        AlgorithmFamily("多智能体强化学习", ("IQL", "VDN", "QMIX", "COMA", "MADDPG", "MAPPO", "MASAC"), "局部观测→联合交互→集中训练/分散执行", "多AGV协同与通信", ("MARLDecisionAdapter", "SafetyShield")),
        AlgorithmFamily("分层强化学习", ("Options", "MAXQ", "Feudal", "Manager-Worker"), "高层选子目标→低层执行→跨层奖励", "任务分解、派工与路径的多时间尺度", ("HybridCoordinator", "RLDecisionAdapter")),
        AlgorithmFamily("安全/约束/离线/模仿", ("Action Mask", "Shield", "CMDP", "CPO", "BC", "GAIL", "CQL", "IQL"), "约束/数据→安全学习→部署前验证", "不能在线冒险、已有历史日志", ("ConstraintChecker", "SafetyShield", "ReplayBuffer")),
        AlgorithmFamily("博弈/拍卖/市场", ("势博弈", "演化博弈", "拍卖", "合同网"), "参与者→效用→响应/竞价→均衡/分配", "分布式资源竞争和能量均衡", ("ObjectiveEvaluator", "MARLDecisionAdapter")),
        AlgorithmFamily("感知/预测/表征", ("YOLO", "CNN", "LSTM", "GRU", "Transformer", "GNN"), "原始数据→特征/预测→交给决策器", "行人、拥堵、图结构和时序预测", ("StateBuilder", "RLDecisionAdapter")),
    )


# ============================================================================
# 第 8 层：可直接运行的最小示例
# ============================================================================


def build_demo() -> tuple[ProblemDefinition, SchedulingSnapshot]:
    graph = WeightedGraph()
    graph.add_edge("S", "P1", 2)
    graph.add_edge("S", "P2", 4)
    graph.add_edge("P1", "D1", 3)
    graph.add_edge("P2", "D1", 2)
    graph.add_edge("D1", "CHARGE", 2)
    problem = ProblemDefinition(graph=graph)
```

```
    snapshot = StateBuilder().build(
        time=0,
        agvs=(AGV("AGV-01", "S", 90, 2), AGV("AGV-02", "P2", 65, 1)),
        tasks=(
            Task("T-urgent", "P1", "D1", 1, priority=5, due_time=8),
            Task("T-normal", "P2", "D1", 1, priority=1, due_time=12),
        ),
    )
    return problem, snapshot


def main() -> None:
    problem, snapshot = build_demo()
    coordinator = HybridCoordinator(RuleDecisionEngine())
    manager = ExperimentManager(coordinator)
    final_state, metrics, plans = manager.run_until_empty(problem, snapshot)

    print("AGV 智能调度可复用母模块 V2: 运行示例")
    print("-" * 58)
    for i, plan in enumerate(plans, 1):
        print(f"决策 {i}: {plan.agv_id} 执行 {plan.task_id}")
        print(f"  路径: {' -> '.join(plan.full_path)}")
        print(f"  距离: {plan.distance:.1f}; 等待: {plan.wait_steps}; 安全: {plan.safe}")
        print(f"  原因: {plan.explanation}")
    print("-" * 58)
    print(f"完成任务: {metrics.completed_tasks}")
    print(f"总距离: {metrics.total_distance:.1f}")
    print(f"最终时刻: {final_state.time}")


if __name__ == "__main__":
    main()
```